SAT solving for Python and Rust programmers

A tutorial for people who know boolean logic
and have never used a solver

Who this is for. You can read `(a and not b) or c` and say whether it is true for given inputs. You have never used a SAT solver, and you are not sure why you would. This assumes nothing else.

What you will be able to do. Turn a real problem — scheduling, colouring, configuration, puzzle — into something a solver answers in milliseconds, and know when the answer can be trusted.

Every code block here was executed, and every output is what it actually printed.

Contents

Part I — The theory you need, and no more

1 The problem

A *SAT problem* asks one question:

> Given a boolean formula, is there any assignment of true/false to its variables that makes the whole thing true?

That is it. If yes, the formula is **satisfiable** (SAT) and the solver hands you an assignment. If no, it is **unsatisfiable** (UNSAT).

This sounds like a toy. It is not, for one reason: an enormous number of practical problems are *disguised* SAT problems. Scheduling, routing, hardware verification, package dependency resolution, timetabling, puzzle solving — all of them reduce to "find an assignment satisfying these constraints". SAT solvers have become good enough that translating your problem into boolean logic and handing it over often beats writing a bespoke algorithm.

2 Why you would install this

Two different answers, because the two packages are for different jobs. Read the one that applies to you.

If you write Python: `cdclkit`

You reach for this when you have a pile of constraints and need any assignment satisfying all of them. Concretely:

- **Rostering and scheduling.** Six nurses, three shifts, nobody works two nights in a row, everyone gets a weekend off a month.
- **Configuration.** Which package versions can coexist. Which hardware options are mutually compatible.
- **Assignment.** Exams to rooms and time slots with no clashes. Frequencies to transmitters that must not interfere.
- **Puzzles and games.** Sudoku, nonograms, Einstein's zebra — genuinely the same shape of problem as the three above.

The alternative is writing a backtracking search yourself. That is slow to write, and the version you write in an afternoon will be far slower than a solver that has had thirty years of engineering poured into the same loop. Encoding to SAT hands that part to someone else.

Two things you get that a hand-written search does not:

- **When there is no solution, you learn why.** A minimal unsatisfiable subset (Section 13) names the handful of constraints that actually conflict, rather than reporting "infeasible" and leaving you to bisect a thousand of them by hand.
- **When it says "no", you can check it.** That is the whole of Section 5, and it is unusual — most tools ask you to take "no solution exists" on faith.

When not to. If your problem is numeric optimisation — maximise profit subject to linear constraints over real numbers — you want a linear-programming or MIP solver, not this. If you

need raw throughput on industrial instances with millions of clauses, install `python-sat`, which ships kissat and CaDiCaL as binary wheels. `cdclkit` is a readable, self-checking, dependency-free implementation; it is not trying to win competitions, and it says so.

If you write Rust: the `dratify` crate

Narrower, and sharper. You install this when you need to **verify a proof of unsatisfiability inside a Rust process**.

That sounds niche until you hit it. Something — a solver in any language, a model checker, a build tool — reported that no solution exists, and you are about to act on that. Ship the hardware. Declare the configuration impossible. Sign off the safety argument. The report is a bare assertion from a large optimised program, and you would like evidence.

Until now your options in Rust were to shell out to `drat-trim` — C you have to compile and invoke as a subprocess — or to trust the solver. This crate is a dependency-free alternative that does RUP and RAT properly and runs in-process.

When not to. If you want to *solve* from Rust, this is not that; use an established solver crate. This checks proofs. The distinction is the point of Section 5.

3 Conjunctive normal form

Solvers do not accept arbitrary formulas. They accept **CNF**: an AND of ORs.

- A **literal** is a variable or its negation: a, $\neg a$.
- A **clause** is an OR of literals: $(a \vee \neg b \vee c)$.
- A **formula** in CNF is an AND of clauses.

$$(a \vee b) \wedge (\neg a \vee b) \wedge (a \vee \neg b)$$

To satisfy the formula you must satisfy *every* clause, and to satisfy a clause you need *at least one* of its literals true. That is the whole semantics.

Is CNF a limitation?

No. Every boolean formula can be converted to CNF, and there is a standard trick (Tseitin encoding) that does it without the exponential blow-up the naive conversion would cause. It works by naming subexpressions: introduce a fresh variable t standing for $(x \wedge y)$, add clauses forcing $t \leftrightarrow (x \wedge y)$, and use t from then on.

You rarely do this by hand. The library does it for you — Section 11 shows the interface.

4 DIMACS, the file format

Everyone in this field exchanges CNF in one plain-text format, and you will see it constantly.

```
p cnf 2 3
 1  2 0
-1  2 0
 1 -2 0
```

- `p cnf 2 3` — header: 2 variables, 3 clauses.

- Variables are numbered from **1**. A negative number means negation.
- Each clause is a list of literals terminated by `0`.
- Lines starting with `c` are comments.

So the file above is exactly $(a \vee b) \wedge (\neg a \vee b) \wedge (a \vee \neg b)$, with $a = 1$ and $b = 2$.

5 The asymmetry that matters

Here is the thing that makes this field interesting, and it is worth pausing on.

A "yes" answer checks itself. If the solver says satisfiable and hands you an assignment, you verify it in linear time: walk the clauses, check each has a true literal. You do not have to trust the solver at all. You do not even have to understand it.

A "no" answer does not. If the solver says unsatisfiable, what has it given you? Nothing you can check. It is asserting that *none* of the 2^n assignments works. Re-deriving that yourself means redoing the search.

This is not paranoia. Modern SAT solvers are tens of thousands of lines of aggressively optimised C, and bugs in them are found regularly — including in solvers that have won competitions. If your hardware verification tool reports "no counterexample exists", you would like that to mean something.

6 Proofs of unsatisfiability

The field's answer is that the solver must **show its work**.

While searching, a solver learns new clauses that are consequences of the original formula. If it can eventually derive the **empty clause** — a clause with no literals, which cannot be satisfied by anything — then the formula is unsatisfiable. Logging that sequence of derived clauses is a **DRAT proof**.

A separate program, a *checker*, replays the log and verifies each step follows from what came before. The checker is small enough to audit, and shares no code with the solver.

RUP, informally

The main proof rule is **RUP** (reverse unit propagation). To justify a new clause C:

1. Assume C is false, which forces every literal in it to be false.
2. Propagate: any clause with all-but-one literal false forces its remaining literal.
3. If that produces a contradiction, C must have been true. Accept it.

There is a stronger rule, **RAT**, which permits clauses that are not strictly implied but preserve satisfiability. It is what the "A" in DRAT stands for. You do not need it to use a solver; you need it if you write a checker.

7 What a solver actually does

You do not need this to use one, but a paragraph helps.

Modern solvers use **CDCL** — conflict-driven clause learning. Roughly: pick an unassigned variable and guess a value; propagate the forced consequences; if you hit a contradiction, analyse

why and derive a new clause that forbids the mistake; backtrack and continue. The learned clauses accumulate, and each one prunes a chunk of the search space. That learning step is what separates CDCL from brute force, and it is why solvers handle problems with millions of variables.

Part II — Python

8 Installing

```
pip install cdclkit             # solver, encodings, modelling layer
pip install "cdclkit[native]"   # the same, plus a Rust engine ~18x
    faster
```

`cdclkit` depends only on `dratify`, the proof checker, which has no dependencies of its own. Nothing third-party is pulled in.

9 Your first formula

```
"""Smallest possible: write a formula by hand, ask whether it can be
    satisfied."""
from cdclkit import parse_dimacs, solve

# (a OR b) AND (NOT a OR b) AND (a OR NOT b)
# DIMACS: variables are 1, 2, ...; a negative number is a negated
    variable;
# each clause ends with 0.
text = """
p cnf 2 3
 1  2 0
-1  2 0
 1 -2 0
"""

formula = parse_dimacs(text)
print("variables:", formula.nvars, " clauses:", len(formula.clauses))

sat, model = solve(formula)
print("satisfiable:", sat)
if sat:
    # model is indexed by variable number - 1
    print("a =", model[0], " b =", model[1])
```

```
variables: 2  clauses: 3
satisfiable: True
a = True  b = True
```

`solve` returns a *tuple*. Note the shape — `if solve(f):` is always true, because a two-element tuple is always truthy. Unpack it.

10 Unsatisfiable, and proving it

Forbid all four combinations of two variables and nothing is left.

```
"""An unsatisfiable formula, and the proof that it is."""
from cdclkit import parse_dimacs, solve, MemoryProof
import dratify

# All four combinations of two variables are forbidden, so nothing is
    left.
formula = parse_dimacs("""
```

```
p cnf 2 4
 1  2 0
 1 -2 0
-1  2 0
-1 -2 0
""")

proof = MemoryProof()
sat, model = solve(formula, proof=proof)
print("satisfiable:", sat)
print("proof steps:", proof.n_add, "additions,", proof.n_del, "
    deletions")

result = dratify.check_proof(formula, proof)
print("proof verified by an independent checker:", result.ok)
print("empty clause derived:", result.reached_empty)
```

```
satisfiable: False
proof steps: 2 additions, 0 deletions
proof verified by an independent checker: True
empty clause derived: True
```

This is Section 5 made concrete. The solver said "no" and handed over a proof; `dratify` — a different package, sharing no solver code — replayed it and agreed. You did not have to trust the solver.

11 Constraints, without writing clauses

Real problems are not phrased as clauses. They are phrased as "exactly one of these", "at most three of those". The `Encoder` turns those into clauses for you.

```
"""Constraints people actually have, without writing clauses by hand.

Colour a triangle with 2 colours. Each node needs exactly one colour,
    and
adjacent nodes must differ. A triangle needs 3 colours, so this is
    UNSAT.
"""
from cdclkit import CNF, Encoder, solve, neg

for n_colours in (2, 3):
    f = CNF()
    enc = Encoder(f)

    nodes = ["x", "y", "z"]
    edges = [("x", "y"), ("y", "z"), ("x", "z")]

    # colour[node][c] is true when `node` has colour c
    colour = {n: [enc.new_lit() for _ in range(n_colours)] for n in
        nodes}

    for n in nodes:
        enc.exactly_one(colour[n])          # one colour per node
    for a, b in edges:
        for c in range(n_colours):          # never the same colour
            on an edge
            # note neg(), not -x: internal literals are non-negative
```

```
            enc.add([neg(colour[a][c]), neg(colour[b][c])])

    sat, model = solve(f)
    print(f"{n_colours} colours: {'SAT' if sat else 'UNSAT'}"
          f"   ({f.nvars} vars, {len(f.clauses)} clauses)")
```

```
2 colours: UNSAT   (6 vars, 12 clauses)
3 colours: SAT   (9 vars, 21 clauses)
```

A triangle needs three colours, and the solver proves two are impossible.

One gotcha. Internal literals are non-negative — a variable v is $2v$ positive and $2v{+}1$ negated. So negation is `neg(x)`, not `-x`. This differs from DIMACS on purpose: it makes array indexing direct in the solver's hot loops. Use `from_dimacs` when converting signed input.

The encoder also offers `at_most_one`, `at_least_k`, `exactly_k`, pseudo-boolean constraints (`assert_pb_leq`), and Tseitin gates (`and_gate`, `or_gate`, `xor_gate`, `ite`).

12 The modelling layer

For most problems you should not touch literals at all. The modelling layer gives you finite-domain integers.

```
"""The modelling layer: integers and all-different, no clauses in
    sight.

A 4x4 Sudoku. Each cell holds 1..4; rows, columns and 2x2 boxes are
all-different.
"""
from cdclkit.model import Model

PUZZLE = [
    [1, 0, 0, 0],
    [0, 0, 3, 0],
    [0, 4, 0, 0],
    [0, 0, 0, 2],
]

m = Model()
cell = [[m.int_var(range(1, 5), f"c{r}{c}") for c in range(4)] for r
    in range(4)]

for r in range(4):
    m.all_different(cell[r])                                    # rows
for c in range(4):
    m.all_different([cell[r][c] for r in range(4)])             #
        columns
for br in (0, 2):
    for bc in (0, 2):                                           # 2x2
        boxes
        m.all_different([cell[br + i][bc + j] for i in range(2) for j
            in range(2)])

for r in range(4):
    for c in range(4):
        if PUZZLE[r][c]:
```

```
            m.add(cell[r][c] == PUZZLE[r][c])                    # the
                givens

sol = m.solve()
print("solved:", sol is not None)
for r in range(4):
    print("   ", " ".join(str(sol.value(cell[r][c])) for c in range
        (4)))
```

```
solved: True
    1 3 2 4
    4 2 3 1
    2 4 1 3
    3 1 4 2
```

`int_var` takes a domain, `all_different` does what it says, and `cell == 3` builds a constraint. Underneath it is still CNF and still the same solver — but you never see a clause.

This is the layer to reach for first. Drop to the encoder when you need a constraint it does not offer, and to raw clauses almost never.

13 Why is it unsatisfiable?

When a model has no solution, the useful question is which constraints conflict. A **minimal unsatisfiable subset** answers it.

```
"""Why is it unsatisfiable? Ask for the part that matters.

A minimal unsatisfiable subset (MUS) is a subset of the clauses that
    is still
unsatisfiable, and stops being so if you drop any one of them.
"""
from cdclkit import parse_dimacs, mus

# Clause 4 is irrelevant to the contradiction between 1, 2 and 3.
formula = parse_dimacs("""
p cnf 3 4
 1 0
-1 2 0
-2 0
 3 0
""")

core = mus(formula)
print("clause indices in the minimal unsatisfiable subset:", core)
print("dropping any one of them makes it satisfiable again")
```

```
clause indices in the minimal unsatisfiable subset: [0, 1, 2]
dropping any one of them makes it satisfiable again
```

Clause 3 is irrelevant to the contradiction and the MUS excludes it. On a scheduling problem with a thousand constraints, this is the difference between "infeasible" and "these four requirements cannot all hold".

14 The command line

```
cdclkit solve problem.cnf --self-check --check-model
```

Solves, and then: if SAT, re-evaluates the model against the original formula clause by clause; if UNSAT, emits a DRAT proof and has it independently replayed. Exit codes follow the SAT competition convention — `10` for satisfiable, `20` for unsatisfiable — which is unusual enough to surprise a shell script expecting `0`.

Part III — Rust

15 What the Rust crate is for

The `dratify` crate is the *proof checker*, not the solver. That is the piece worth having in Rust: you reach for it when something else produced a DRAT proof and you need to verify it inside a Rust process, without shelling out to a C tool.

If you want to *solve* from Rust, use one of the established solvers. If you want to *check*, this is a dependency-free crate that does RUP and RAT properly.

```
cargo add dratify
```

16 Literals

The crate uses the same doubled index as the Python side, and getting this wrong is the most likely first mistake:

DIMACS	meaning	internal
1	a	0
-1	$\neg a$	1
2	b	2
-2	$\neg b$	3

Variable v becomes $2v$ positive, $2v{+}1$ negated. Negation is a single XOR with 1, which is why the encoding exists.

17 Checking a proof

```
//! Checking a DRAT proof from Rust.
//!
//! Literals use a doubled index: variable `v` is `2v` when positive
    and
//! `2v + 1` when negated. So DIMACS `1` is literal `0`, DIMACS `-1`
    is `1`,
//! DIMACS `2` is `2`, DIMACS `-2` is `3`.

use dratify::{Checker, Lit};

/// Convert a DIMACS literal (1-based, signed) into the internal
    encoding.
fn lit(d: i32) -> Lit {
    let v = (d.unsigned_abs() - 1) as Lit;
    (v << 1) | if d < 0 { 1 } else { 0 }
}

fn main() {
    // (a v b) (a v ~b) (~a v b) (~a v ~b)  -- unsatisfiable
    let formula: Vec<Vec<Lit>> = vec![
        vec![lit(1), lit(2)],
        vec![lit(1), lit(-2)],
        vec![lit(-1), lit(2)],
        vec![lit(-1), lit(-2)],
    ];
```

```
    // A DRAT proof: derive "a", then the empty clause. Each step is
    // (is_deletion, literals).
    let proof: Vec<(bool, Vec<Lit>)> = vec![
        (false, vec![lit(1)]),
        (false, vec![]),
    ];

    let mut checker = Checker::new(2, &formula, true, true);
    let r = checker.check(&proof);

    println!("verified:      {}", r.ok);
    println!("empty clause:  {}", r.reached_empty);
    println!("steps checked: {} ({} by RUP, {} by RAT)", r.steps, r.
        rup_steps, r.rat_steps);

    // Now a dishonest proof: claim the empty clause with nothing to
        back it.
    let bogus: Vec<(bool, Vec<Lit>)> = vec![(false, vec![])];
    let mut c2 = Checker::new(2, &vec![vec![lit(1)]], true, true);
    let bad = c2.check(&bogus);
    println!();
    println!("bogus refutation accepted: {}", bad.ok);
    println!("reason: {}", bad.reason);
}
```

```
verified:      true
empty clause:  true
steps checked: 2 (2 by RUP, 0 by RAT)

bogus refutation accepted: false
reason: clause is neither RUP nor RAT with respect to the clauses
    available
         at this point
```

The second half matters more than the first. A checker that accepts everything would pass every test that only feeds it valid proofs. Any checker you rely on should be tested on proofs that are *wrong*, and should reject them for the right reason.

18 What the result tells you

`check` returns a `CheckResult`:

`ok`	the proof is a valid refutation
`reached_empty`	the empty clause was actually derived
`steps`, `rup_steps`, `rat_steps`	what was checked, and how
`failed_step`, `failed_clause`	where it went wrong, if it did
`reason`	why, in words

Note that `ok` and `reached_empty` are separate. A proof whose every step verifies but which never derives the empty clause is not a refutation — it is a valid but incomplete derivation. Conflating the two is how a truncated proof gets accepted.

19 Two implementations on purpose

The Python package ships its own pure-Python checker, and this crate is an independent implementation of the same rules. Neither is the "real" one.

For a proof checker that is the entire point: agreement between independent implementations is the evidence. They are differentially tested against each other on rejections as well as acceptances, which is the half that catches a checker that has quietly started saying yes to everything.

From Python you can have both. `pip install "cdclkit[native]"` installs wheels that register the Rust checker with `dratify`, and `check_proof(..., engine="python")` versus `engine="native"` then runs each.

Where to go next

- `examples/` in the `cdclkit` repository — Sudoku with a uniqueness proof, the zebra puzzle, graph colouring, bounded model checking, circuit equivalence.
- `docs/ALGORITHMS.md` — the mathematics, from resolution through first-UIP learning, LBD, DRAT and preprocessing, including a section on what is deliberately absent.
- For industrial-scale problems, `python-sat` bundles kissat and CaDiCaL as binary wheels. This project is a readable, self-checking implementation; it is not trying to win competitions.